

catastrophic forgetting.

Full fine-tuning can be used:

- When the domain is very different from the general knowledge of the model (e.g., biomedical, aerospace domains).
- When maximum accuracy and deep adaptation are needed.
- When compute resources and large training datasets are available.

Note:

- **Overfitting:** The model may memorise training examples instead of learning general patterns if the dataset is too narrow/small. The model performs perfectly on the training examples but loses its ability to handle new and unseen data.
- **Catastrophic forgetting:** The model may lose general capabilities because new training overwrites pre-training knowledge.

Parameter-efficient fine-tuning

(PEFT): Parameter-efficient fine-tuning (PEFT) updates only a small subset of the model's parameters while keeping the base pre-trained model frozen. This approach significantly reduces training costs, GPU memory usage, and the overall time required for fine-tuning.

Instead of modifying billions of parameters, PEFT techniques add lightweight trainable components, such as low-rank matrices, adapter layers, or learned prompts to the existing model. These components learn task-specific behaviour while preserving the general knowledge already present in the base model. As a result, PEFT makes fine-tuning practical even on limited hardware and is widely adopted in the open source community.

PEFT can be used:

- When compute resources are limited (single GPU or consumer-grade hardware).
- When multiple task-specific models need to be derived from a single base LLM.

- When fast experimentation and iteration are required.
 - When preserving the base model's general knowledge is important.
- Some commonly used PEFT techniques include LoRA (Low-Rank Adaptation) and QLoRA (Quantized LoRA).

LoRA (Low-Rank Adaptation):

LoRA introduces small trainable low-rank matrices into selected Transformer layers. During training, only these added matrices are updated, while the original model weights remain frozen. This greatly reduces the number of trainable parameters without significantly affecting model performance. Use LoRA when you need general enterprise fine-tuning with moderate domain data and want high accuracy at low training cost.

This is how LoRA works internally:

- During fine-tuning, the base model weights remain frozen (W_0), and LoRA learns only a small task-specific update (ΔW). The updated weight used by the model is: $W = W_0 + \Delta W$
- Instead of learning a large $d \times d$ update matrix, LoRA represents the update in a compact low-rank form: $\Delta W = BA$

Here:

d = the model's hidden size (its internal feature dimension)

A = a small matrix with r rows and d columns

B = a small matrix with d rows and r columns

r = the low rank (a small number compared to d ; typically, 4, 8, or 16)

- This reduces trainable parameters from d^2 to $2dr$, making training faster, cheaper, and more memory-efficient while maintaining strong accuracy.

QLoRA: QLoRA extends LoRA by loading the base model in 4-bit quantized precision, which further reduces GPU memory usage. The LoRA components remain in higher precision and are trained normally. This combination enables fine-tuning of very large models (30B-70B parameters) on a single GPU. Use QLoRA when training large models (13B-70B) on limited hardware; 4-bit quantization reduces memory while preserving model quality.

Note:

- **Low-rank matrix:** A compact mathematical representation that approximates large weight updates using much smaller matrices, reducing memory and computation.
- **Quantization (4-bit):** A technique that stores model weights using fewer bits (4 instead of 16 or 32), lowering memory usage with minimal loss in accuracy.

This first article in the two-part series has established the foundation for understanding how and why large language models must be adapted for real-world, domain-specific use cases. With this conceptual grounding, the next part will demonstrate a complete hands-on workflow for fine-tuning an open source LLM (Mistral-7B-Instruct) using PEFT QLoRA in Google Colab environment. 

By: Subranshu Patel and Shwetabh Srivastava

Subranshu Patel is a group manager in the system software division of HCL Technologies Limited. He has extensive experience in Linux-based OS and ecosystem design and development, and in open source technologies.

Shwetabh Srivastava is a solutions architect at HCL Technologies Limited. He is experienced in designing and developing Linux-based system software, with expertise in operating system principles and the inner workings of the Linux kernel.